

NFStream CICFlowmeter Plugins Documentation

1. HeaderLenPlugin

Features Created:Fwd Header Len, Bwd Header Len

Description

This plugin calculates the cumulative length of all network headers (IP layer + Transport layer) across the entire lifespan of a flow. It tracks these header sums independently for both the forward (source-to-destination) and backward (destination-to-source) directions.

Logic

Core Calculation: For any given packet, the exact header size is calculated dynamically by subtracting the pure data payload size from the total IP packet size (`packet.ip_size - packet.payload_size`). There isn't a `.header_len()` attribute available for the NFPacket object so this was the only calculation that was possible.

Initialization (`on_init`): Intercepts the very first packet of the flow. It calculates the header size, assigns it to the correct directional counter (forward or backward), and initializes the opposite counter to 0.

Update Logic (`on_update`): For every subsequent packet, it calculates the header size and adds it to the running total of the respective direction.

2. InitWindowPlugin

Features Created:Init Fwd Win Byts, Init Bwd Win Byts

Description

This plugin captures the initial TCP window sizes advertised by both the client and the server at the beginning of a TCP connection. It specifically targets the first SYN packet (forward direction) and the corresponding SYN-ACK packet (backward direction) to extract the raw window values before dynamic window scaling alters them.

Logic

Initialization (**on_init**):

1. Sets default values to **-1** to handle non-TCP traffic or incomplete handshakes.
2. Intercepts the very first packet of the flow. If it is a TCP packet (Protocol 6), it checks the flags.
3. If the packet is a pure **SYN** (client starting connection), it extracts the forward window size.
4. If the packet is a **SYN-ACK** (e.g., if the capture started mid-handshake), it extracts the backward window size. (Handling edge cases.)

Update Logic (**on_update**):

1. Ignores non-TCP traffic.
2. Continuously checks incoming packets for the presence of the **SYN** or **SYN-ACK** flags.
3. Once it successfully extracts a window size for a direction, it locks that value (ignoring any subsequent retransmissions or flags) to ensure only the *initial* window is recorded.

The NFPacket object doesn't have any attributes for getting the initial window size directly. But it has an attribute called **.ip_packet()** which contains raw content starting from IP Header. The following helper functions uses this above attribute.

Extraction Helper (**_extract_tcp_window**):

1. Accesses the raw byte array of the packet.
2. Calculates the dynamic IP header length (handling both IPv4 and IPv6; for IPv4 its packet size is present in the last four bits of first byte*4. For IPv6 it's set to 40) to accurately locate the start of the TCP header.
3. Reads the 2-byte window size located at offset 14 of the TCP header using bitwise shifting (**<< 8**) and logical OR (**|**). (The TCP window size is 2 bytes, located at offset 14 of the TCP header)

3. BulkPlugin

Features Created: Fwd Byts/b Avg, Fwd Pkts/b Avg, Bwd Byts/b Avg, Bwd Pkts/b Avg, Fwd Blk Rate Avg, Bwd Blk Rate Avg

Description

This plugin identifies and tracks "bulk data transfer" phases within a network flow. A bulk transfer is defined as a continuous sequence of data transmission in a single direction that exceeds a 4-packet threshold without being interrupted by a long timeout or an opposite-direction payload. It calculates the average byte rates, packet rates, and overall bulk transfer speeds for both the forward and backward directions.

Logic

Initialization (on_init): Initializes "helper" variables(to track the needed conditions) and "bulk" variables. It explicitly passes the very first packet into the update logic to ensure UDP/ICMP payloads aren't missed.

Update Logic (on_update):

1. Ignore empty packets (payload = 0).
2. Checks for sequence interruptions: If more than 1000ms has passed, or if the opposite direction sent a payload, the "helper" waiting room resets to 0.
3. If uninterrupted, the packet is added to the waiting room.
4. **The 4-Packet Threshold:** Once the waiting room hits exactly 4 packets, the sequence is officially upgraded to a "Bulk Phase," and the totals are committed. Any further uninterrupted packets in that direction are added directly to the bulk totals.

Expiration (on_expire): Divides the committed bulk totals by the number of bulk phases to calculate the final averages (Bytes per Bulk, Packets per Bulk, and Bulk transfer rate in Bytes/sec).

4. ActiveIdlePlugin

Features Created:Active Mean/Min/Std/Max, Idle Mean/Min/Std/Max

Description

This plugin tracks the duration of "Active" and "Idle" phases within a network flow. A flow is considered Active as long as packets are being continuously exchanged. If a period of silence exceeds a specific timeout threshold (defaulting to 5 seconds / 5000ms), the

flow transitions to an Idle state. The plugin calculates the mean, standard deviation, maximum, and minimum durations for both of these states across the flow's lifecycle.

Logic

Initialization (`on_init`): Sets the `_start_active_time` and `_end_active_time` to the timestamp of the first packet and prepares the baseline tracking variables.

State Tracking (`on_update`): For every subsequent packet, it calculates the time gap between the current packet and the last seen packet (`_end_active_time`).

- **Active State (`Gap <= 5000ms`):** The flow is considered to be in continuous communication. The plugin simply pushes the `_end_active_time` forward to the current packet's timestamp.
- **Idle State (`Gap > 5000ms`):** A period of silence has breached the threshold. The plugin finalizes the previous Active phase, calculates its total duration, and records the gap itself as an Idle duration. It then resets the timers for a brand-new Active phase.

Expiration (`on_expire`): When the flow terminates, it closes out the final pending Active phase and outputs the final metrics (Mean, Std Dev, Max, Min).

To calculate the mean and the standard deviation without storing the value's time value in an infinite list, [Welford's online algorithm](#) was used.

5. ExtraFeaturesPlugin

Features Created: Fwd Act Data Pkts & Fwd Seg Size Min

Description

This plugin extracts two specific metadata points from the forward (source-to-destination) direction of a flow: the total count of packets that carry actual application data (ignoring empty control packets), and the absolute minimum header size observed during the flow.

Logic

Initialization (`on_init`):

1. Sets the data packet counter (`fwd_act_data_pkts`) to 0.

2. Initializes the minimum segment size tracker (`fwd_seg_size_min`) to `-1` to act as a placeholder, ensuring the first observed packet correctly overwrites it.
3. Immediately passes the first packet into the core update helper function.

Core Update Logic (`_update_features`):

1. **Direction Filter:** The plugin strictly filters for forward traffic (`packet.direction == 0`), entirely ignoring backward packets.
2. **Active Data Packets:** It checks the `payload_size`. If the payload is strictly greater than 0, it increments the `fwd_act_data_pkts` counter. (This successfully ignores empty `SYN`, `FIN`, or pure `ACK` packets).
3. **Minimum Segment Size (Header Length):** It calculates the dynamic header length of the packet (`packet.ip_size - packet.payload_size`). If this is the first packet (tracked via `-1`) or if this header is smaller than the currently stored minimum, it updates `fwd_seg_size_min`.

(Note: Although the name suggests segment size internally CICFlowmeter uses header length).